Homework 6

By Andrew McLaughlin

Problem 1

Let S be the column vector with components $S^{[1]}, S^{[2]}$, where the stock prices $S^{[j]}$ have risk-neutral dynamics

$$dS_t^{[j]} = rS_t^{[j]}\,dt + \sigma^{[j]}S_t^{[j]}\,dW_t^{[j]}, \qquad j = 1, 2$$

with risk-free interest rate $r = 0.05$, and constant volatilities $\sigma^{[1]} = 0.3$, $\sigma^{[2]} = 0.2$. The time-0 prices are $S_0^{[1]} = 100$, $S_0^{[2]} = 110$. The P-Brownian motions $W^{[1]}$ and $W^{[2]}$ have correlation $\rho = 0.8$.

(a)

Let X be the column vector with components $X^{[1]}, X^{[2]}$ where $X^{[j]} := \log S^{[j]}$. Find the covariance matrix of X_T.

Hint: One approach is to manually fill in the covariance matrix, using relationships such as

$$\mathrm{Cov}(W_T^{[1]}, W_T^{[2]}) = 0.8T$$

in combination with the volatilities.

Another approach is to use matrix multiplication: write X_T as a nonrandom vector plus ΣW_T where Σ is the nonrandom diagonal matrix with diagonal elements $\sigma^{[1]}, \sigma^{[2]}$, and W is the random column vector with components $W^{[1]}, W^{[2]}$. Then

$$\mathrm{Cov}(X_T) = E(\Sigma W_T W_T^\top \Sigma^\top) = \Sigma\,\mathrm{Cov}(W_T)\Sigma^\top = T\Sigma\,\mathrm{Corr}(W_T)\Sigma^\top.$$

Consider a basket

$$H := \frac{1}{2}S^{[1]} + \frac{1}{2}S^{[2]}$$

of one-half of a share of each stock.

Answer: (work on pdf)

$$\mathrm{Cov}(X_T) = T\begin{pmatrix} \sigma_1^2 & \rho\sigma_1\sigma_2 \\ \rho\sigma_1\sigma_2 & \sigma_2^2 \end{pmatrix} = T\begin{pmatrix} 0.09 & 0.048 \\ 0.048 & 0.04 \end{pmatrix}$$

(b)

Using 10000 standard Monte Carlo simulations, estimate the time-0 price C of an option that pays $(H_T - 110)^+$ at time $T = 1.0$. Also give the standard error [the sample standard deviation, divided by the square root of the number of simulations] of your Monte Carlo estimate.

You may either use a random number generator that produces normals with a given covariance matrix (which you found in (a)), or alternatively use a random number generator that produces independent normals which you then transform to introduce correlation.

In either approach, each of the 10000 simulations should use just one $\mathbb{R}^2$-valued random vector Z of simulated normal zero-mean random variables.

In [10]:
```
import numpy as np
```

In [11]:
```
class MultiGBM:

    def __init__(self,S0,r,correlations,sigma):
        self.S0 = S0
        self.r = r
        self.correlations = correlations
        self.sigma = sigma
```

In [12]:
```
hw6p1dynamics = MultiGBM(S0=np.array([100,110]),r=0.05,
                         correlations = np.array([[1,0.8],[0.8,1]]),
                         sigma = np.diag([0.3, 0.2]))
```

In [13]:
```
class CallOnBasket:

    def __init__(self,K,T,weights):
        self.K = K
        self.T = T
        self.weights = weights
```

In [14]:
```
hw6p1contract=CallOnBasket(K=110,T=1.0,weights = np.array([1/2, 1/2]))
```

In [15]:
```
class MCengine:

    def __init__(self, M, antithetic, control, seed):
        self.M = M                                  # How many simulations
        self.antithetic = antithetic
        self.control = control
        self.rng = np.random.default_rng(seed=seed) # Seeding the random number generator with a specified number hel

    def price_callonbasket_multiGBM(self, contract, dynamics):

        # You complete the coding of this function.
        # self.rng.multivariate_normal may be useful.
        # See documentation for numpy.random.Generator.multivariate_normal
        # as self.rng is an instance of numpy.random.Generator

        # You are not required to support the case where MC.control = MC.antithetic = True
        # (simultaneous use of control variate and antithetic)
        # But you are required to support the other 3 possible settings of MC.antithetic/MC.control
        # namely False/False, True/False, False/True.
        # (ordinary MC, antithetic without control, control without antithetic)

        S0 = dynamics.S0
        r = dynamics.r
        Sigma = dynamics.sigma                      # diagonal vol matrix
        sigma_vec = np.diag(Sigma)                  # length-2 vol vector
        corr = dynamics.correlations
        K = contract.K
        T = contract.T
        w = contract.weights
        M = self.M
        disc = np.exp(-r * T)

        # Cov(X_T) from part (a): T * Sigma @ Corr @ Sigma^T
        cov = T * (Sigma @ corr @ Sigma.T)
        # Mean of X_T = log S_T under the risk-neutral measure
        mean = np.log(S0) + (r - 0.5 * sigma_vec**2) * T

        if self.antithetic:
            Z = self.rng.multivariate_normal(mean=np.zeros(2), cov=cov, size=M)
            S_plus  = np.exp(mean + Z)
            S_minus = np.exp(mean - Z)
            payoff_plus  = np.maximum(S_plus  @ w - K, 0.0)
```

```
            payoff_minus = np.maximum(S_minus @ w - K, 0.0)
            pair_mean = 0.5 * disc * (payoff_plus + payoff_minus)
            call_price = pair_mean.mean()
            standard_error = pair_mean.std(ddof=1) / np.sqrt(M)
        elif self.control:
            from scipy.stats import norm
            Z = self.rng.multivariate_normal(mean=np.zeros(2), cov=cov, size=M)
            X_T = mean + Z                          # log S_T samples, shape (M, 2)
            S_T = np.exp(X_T)
            H_T = S_T @ w                           # arithmetic basket
            G_T = np.exp(X_T @ w)                   # geometric basket: log G = w . log S

            payoff_H = disc * np.maximum(H_T - K, 0.0)
            payoff_G = disc * np.maximum(G_T - K, 0.0)

            # Exact time-0 price of the geometric-basket call.
            # log G_T ~ N(m_G, v_G) under Q; price = e^{-rT} E[(e^{logG_T} - K)^+].
            m_G = w @ mean                          # E[log G_T]
            v_G = w @ cov @ w                       # Var(log G_T)
            sd_G = np.sqrt(v_G)
            d1 = (m_G + v_G - np.log(K)) / sd_G
            d2 = d1 - sd_G
            C_G_exact = (np.exp(-r * T + m_G + 0.5 * v_G) * norm.cdf(d1)
                         - K * np.exp(-r * T) * norm.cdf(d2))

            # Estimated-beta control variate (L6.14):
            # C_hat = mean(H) - beta_hat * (mean(G_MC) - C_G_exact)
            cov_HG = np.cov(payoff_H, payoff_G, ddof=1)
            beta_hat = cov_HG[0, 1] / cov_HG[1, 1]
            Y_cv = payoff_H - beta_hat * (payoff_G - C_G_exact)
            call_price = Y_cv.mean()
            standard_error = Y_cv.std(ddof=1) / np.sqrt(M)
        else:
            Z = self.rng.multivariate_normal(mean=np.zeros(2), cov=cov, size=M)
            S_T = np.exp(mean + Z)
            payoff = np.maximum(S_T @ w - K, 0.0)
            discounted = disc * payoff
            call_price = discounted.mean()
            standard_error = discounted.std(ddof=1) / np.sqrt(M)

        return(call_price, standard_error)
```

In [16]:
```
hw6p1bMC=MCengine(M=10000,antithetic=False,control=False,seed=0)
(call_price_ordinary, std_err_ordinary) = hw6p1bMC.price_callonbasket_multiGBM(hw6p1contract,hw6p1dynamics)
print(call_price_ordinary, std_err_ordinary)
```

```
9.875007332598605 0.16838960569542327
```

(c)

Use 10000 antithetic pairs $(Z, -Z)$ to estimate C, together with a standard error (L6.8).

Consider the "geometric basket"

$$G := \left(S^{[1]} S^{[2]}\right)^{1/2}.$$

In [17]:
```
hw6p1cMC=MCengine(M=10000,antithetic=True,control=False,seed=0)
(call_price_AV, std_err_AV) = hw6p1cMC.price_callonbasket_multiGBM(hw6p1contract,hw6p1dynamics)
print(call_price_AV, std_err_AV)
```

```
9.941127180624436 0.09518450200353228
```

(d)

The random variable $\log G_T$ is normally distributed (because it's a linear transformation of a multivariate normal vector). Show that $\log G_T$ has expectation

$$\frac{1}{2}\log(S_0^{[1]} S_0^{[2]}) + \left(r - \frac{(\sigma^{[1]})^2 + (\sigma^{[2]})^2}{4}\right) T$$

and variance

$$\frac{(\sigma^{[1]})^2 + 2\rho\sigma^{[1]}\sigma^{[2]} + (\sigma^{[2]})^2}{4} T.$$

(e)

Let C_G be the time-0 price of a geometric basket option paying $(G_T - K)^+$ at time T. Express C_G in terms of the function C_{BS} defined in FINM 33000 L6. Specifically, fill in the blanks:

$$C_G = C_{BS}(\_\_\_\_\_, 0, K, T, \_\_\_\_\_, r, \_\_\_\_\_).$$

Your answer should be a general formula, in which you have not substituted 0.8 for ρ, etc. (You may also do the substitutions, but don't neglect the general formula).

(f)

Using a geometric basket option as a control variate, run $M = 10000$ Monte Carlo simulations to estimate C, together with a standard error. Use the control variate estimate $\hat{C}_M^{cv,\hat{\beta}}$ from L6.13 or L6.14. Use the (asymptotically valid) standard error $\hat{\sigma}_M^{cv,\hat{\beta}}/\sqrt{M}$.

See the ipynb file.

In [18]:
```
hw6p1fMC=MCengine(M=10000,antithetic=False,control=True,seed=0)
(call_price_CV, std_err_CV) = hw6p1fMC.price_callonbasket_multiGBM(hw6p1contract,hw6p1dynamics)
print(call_price_CV, std_err_CV)
```

```
9.991036492142777 0.0044010170444346835
```

Problem 2

Each unit of the bank account has price $B_t = e^{rt}$ for all $t \geq 0$.

(a)

Let S_t be the time-t price of a stock that continuously pays a constant proportional dividend yield q. This means that each 1 share of S at time 0 will grow, via dividend reinvestment, to e^{qt} shares of S at each time $t \geq 0$. Thus the stock, divorced from its dividend stream, should not be regarded as a holdable/tradeable asset. Rather, units of the "bundle" should be regarded as holdable/tradeable, where 1 unit of the bundle is defined to be

$$e^{qt} \text{ shares of stock, at all times } t \geq 0.$$

Aside from the above information, do not assume any specific dynamics for S.

By replication, find the time-t value of a forward contract which pays $S_T - K$ at time T, where $t \leq T$. How many shares of S, and what dollar value in the bank account, does the replicating portfolio hold at time t?

Conclude that the time-t forward price for time-T delivery of S_T is

$$F_t = S_t e^{(r-q)(T-t)}.$$

(The forward price is not the same thing as the value of a forward contract.)

(b)

Let S_t be the time-t price of a stock that pays a fixed dollar dividend D discretely at time T_0, where $0 < T_0 < T$. Assume S does not pay any other dividends between time 0 and T.

Let us regard as holdable/tradeable the following bundle. One unit of the bundle consists of:

Holding	Time
1 share	at all times $t < T_0$
1 share plus De^{-rT_0} units of bank account	at all times $t \geq T_0$

Like the case of a continuous proportional dividend yield in (a), the bundle here in (b) absorbs the dividend payments. Unlike (a), the bundle in (b) allocates the dividend into bank account units, not into more stock shares. We are still not assuming any specific dynamics for S.

By replication, find the time-t value of a forward contract which pays $S_T - K$ at time T. How many shares of S, and what dollar value in the bank account, does the replicating portfolio hold at time t? The answer depends on whether $t < T_0$ or $t \geq T_0$.

Conclude that the time-t forward price for time-T delivery of S_T is

$$F_t = \begin{cases} S_t e^{r(T-t)}, & \text{if } t \geq T_0, \\ S_t e^{r(T-t)} - De^{r(T-T_0)}, & \text{if } t < T_0. \end{cases}$$

(The forward price is not the same thing as the value of a forward contract.)

Intuitively, $F_t = E_t S_T$ can be calculated by growing S_t using drift r from today t until delivery date T, combined with growing the $-D$ dollars, from the date T_0 when S gives away the D dollars, until the forward contract's delivery date T. (This does not require comment from you, this is just giving you a different perspective on the replication result.)

Price dynamics of stocks that pay dividends

To prepare for part (c) below: Diffusion/SDE models of price dynamics easily adapt to include continuous dividend yields, but a more realistic dividend model for single stocks is a discrete dividend.

- The dynamics of a stock S that pays out a continuous dividend q can be modeled by adjusting the drift to $r - q$, meaning that the drift term is $(r - q)S_t\,dt$.

  When pricing Americans, the early exercise condition needs to be checked at every time step.

- The dynamics of a stock S that pays a discrete dividend of D dollars to holders at time T_0 can be modeled with a drift of r, along with a down-jump of D dollars at time T_0. But what prevents S from going negative due to the down-jump? A simple approach is to use the SDE to model the forward price dynamics (which will not jump at time T_0), rather than directly modeling the spot S. The initial spot S_0 needs to be converted into a forward price, which evolves according to the SDE. When a payout of an option on S needs to be computed, the forward needs to be converted back into spot. These conversions use the (b) result.

  When pricing Americans, assuming $r \geq 0$, the early exercise condition needs to be checked only at time T_0, immediately prior to the stock dropping D dollars.

(c)

Complete the code in `finm320-26-hw6-p2.ipynb` to find, using finite differences, the time-0 price of a 100-strike 0.25-expiry American call option on a stock S that pays a discrete dividend of 2 dollars at time 0.15, assuming that the time-t forward price X_t for the time-0.25 delivery of S has the CEV dynamics

$$dX_t = 3X_t^{0.5}\,dW_t$$

and $r = 0.05$.

Only two lines of code need to be completed.

The F notation here corresponds to the X notation in the Python code.

In [1]:
```
import numpy as np
from scipy.sparse import diags
from scipy.sparse.linalg import spsolve
```

In [2]:
```
class CEV:

    # Allow X0 to be initially unspecified.  Because we may want to use X to model a forward price X0 that will be ca
    def __init__(self,volcoeff,alpha,rGrow,r,X0=None):
        self.volcoeff = volcoeff
        self.alpha = alpha
        self.rGrow = rGrow
        self.r = r
        self.X0 = X0
```

In [3]:
```
class discreteDividendModel:

    def __init__(self,S0,T,forwardDynamics,discreteDivDate,discreteDivAmt):
        self.discreteDivDate = discreteDivDate
        self.discreteDivAmt = discreteDivAmt
        self.S0 = S0
        self.T = T
        self.forwardDynamics = forwardDynamics
        self.forwardDynamics.X0 = self.convertSpotToForward(S0,0)

    def getX0(self):
        return self.forwardDynamics.X0

    def convertSpotToForward(self,S,t,tBumpToDetectDivdate=1e-12):
        r = self.forwardDynamics.r
        T = self.T
        T0 = self.discreteDivDate
        D = self.discreteDivAmt
        # time t=T0 has two "sides": immediately before and immediately after the transition of S, from including to
        # The sign of tBumpToDetectDivdate indicates which side we are on at time t=T0.
```

```
        if t + tBumpToDetectDivdate > T0:
            return S*np.exp(r*(T-t))
        else:
            return S*np.exp(r*(T-t))-D*np.exp(r*(T-T0))

    def convertForwardToSpot(self,X,t,tBumpToDetectDivdate=1e-12):
        r = self.forwardDynamics.r
        T = self.T
        T0 = self.discreteDivDate
        D = self.discreteDivAmt
        # time t=T0 has two "sides": immediately before and immediately after the transition of S, from including to
        # The sign of tBumpToDetectDivdate indicates which side we are on at time t=T0.
        if t + tBumpToDetectDivdate > T0:
            return X*np.exp(-r*(T-t))
        else:
            return (X+D*np.exp(r*(T-T0)))*np.exp(-r*(T-t))

        # These are the only two lines of code that need to be completed.
        # Hint... do the "opposite" of what convertSpotToForward does
```

In [4]:
```
hw6ForwardDynamics = CEV(volcoeff=3, alpha=-0.5, rGrow=0, r=0.05)
hw6discreteDividendModel = discreteDividendModel(S0=100,T=0.25,forwardDynamics=hw6ForwardDynamics,discreteDivDate=0.1
```

In [5]:
```
class Call:

    def __init__(self,T,K,American=False):
        self.T = T
        self.K = K
        self.American = American
```

In [6]:
```
hw6contract=Call(T=0.25,K=100,American=True)
```

In [7]:
```
class FD_CrankNicolson_Engine:

    def __init__(self,XMax,XMin,deltaX,deltat):
        self.XMax=XMax
        self.XMin=XMin
        self.deltaX=deltaX
        self.deltat=deltat

    def TicksAndMatricesCEV(self,T,dynamics):
```

```
        alpha, r, rGrow, volcoeff = dynamics.alpha, dynamics.r, dynamics.rGrow, dynamics.volcoeff

        N=round(T/self.deltat)
        if abs(N-T/self.deltat)>1e-12:
            raise ValueError('Bad time step')
        numX=round((self.XMax-self.XMin)/self.deltaX)+1
        if abs(numX-(self.XMax-self.XMin)/self.deltaX-1)>1e-12:
            raise ValueError('Bad space step')
        X=np.linspace(self.XMax,self.XMin,numX)    #The FIRST indices in this array are for HIGH levels of X
        tTicks = np.arange(N-1,-1,-1)*self.deltat

        ratio1 = self.deltat/self.deltaX
        ratio2 = self.deltat/self.deltaX**2
        f = (1 / 2) * (volcoeff ** 2) * (X ** (2 * (1 + alpha)))
        g = rGrow * X
        h = -r * np.ones(np.size(X))          ### Scalar also acceptable here
        F = 0.5*ratio2*f + 0.25*ratio1*g
        G =     ratio2*f - 0.50*self.deltat*h
        H = 0.5*ratio2*f - 0.25*ratio1*g

        #Right-hand-side matrix
        RHSmatrix = diags([H[:-1], 1-G, F[1:]], [1,0,-1], shape=(numX,numX), format="csr")

        #Left-hand-side matrix
        LHSmatrix = diags([-H[:-1], 1+G, -F[1:]], [1,0,-1], shape=(numX,numX), format="csr")
        # diags creates SPARSE matrices

        return(X, tTicks, LHSmatrix, RHSmatrix, H[-1], F[0])

    def price_call_CEV(self,contract,discreteDividendModel):

        # returns array of all initial X levels,
        # and the corresponding array of call prices

        if discreteDividendModel.T < contract.T:
            raise ValueError('The forward price cannot be for a delivery date earlier than the expiry of the option')

        dynamics = discreteDividendModel.forwardDynamics

        X, tTicks, LHSmatrix, RHSmatrix, bottomH, topF = self.TicksAndMatricesCEV(contract.T,dynamics)
        # The X array contains the _interior_ levels of the grid,
```

```
        # from the smallest XMin to the largest XMax
        # The boundary conditions are imposed one level _beyond_,
        # e.g. at X_lowboundary=XMin-deltaX, not at XMin.
        # To relate to lecture notation, X_lowboundary is X_{-J}
        # whereas XMin is X_{-J+1}

        callprice=np.maximum(discreteDividendModel.convertForwardToSpot(X,contract.T)-contract.K,0)
        X_highboundary=self.XMax+self.deltaX

        for t in tTicks:

            rhs = RHSmatrix * callprice

            #Now let's add the boundary condition vectors.
            #They are nonzero only in the first component:
            rhs[0]=rhs[0]+2*topF*(discreteDividendModel.convertForwardToSpot(X_highboundary,t)-contract.K*np.exp(-dyr
            #Strictly speaking, this aggregation of boundary conditions at time t and t+dt should have a t term plus
            #But the difference is negligible

            callprice = spsolve(LHSmatrix, rhs)  #You code this.  Hint...
            # numpy.linalg.solve, which expects arrays as inputs,
            # is fine for small matrix equations, and for matrix equations without special structure.
            # But for large matrix equations in which the matrix has special structure,
            # we may want a more intelligent solver that can run faster
            # by taking advantage of the special structure of the matrix.
            # Specifically, in this case, let's try to use a solver that recognizes the SPARSE MATRIX structure.
            # Try spsolve, imported from scipy.sparse.linalg

            if contract.American:
                if abs(t-discreteDividendModel.discreteDivDate)<self.deltat/2:  #if t is the dividend date
                    #then check whether it's better to exercise immediately before the stock price drops by the divia
                    callprice = np.maximum(callprice, discreteDividendModel.convertForwardToSpot(X,t,-self.deltat/2)-

        return(discreteDividendModel.convertForwardToSpot(X,t), X, callprice)
```

In [8]:
```
X0 = hw6discreteDividendModel.getX0()
deltaX = 0.1
hw6FD = FD_CrankNicolson_Engine(XMax=X0+deltaX*1000,XMin=X0-deltaX*500,deltaX=deltaX,deltat=0.0005)
```

In [9]:
```
(S0_all, X0_all, callprice) = hw6FD.price_call_CEV(hw6contract,hw6discreteDividendModel)
```

In [10]:
```
# price_call_CEV gives us option prices for ALL X0 from XMin to XMax
# But let's display only a few rows

import pandas as pd

df = pd.DataFrame({
    'Spot Price (S0)': S0_all,
    'Forward Price (X0)': X0_all,
    'Call Price': callprice
})

df_display = df[(df['Spot Price (S0)'] > hw6discreteDividendModel.S0 - hw6FD.deltaX * 1.5)
              & (df['Spot Price (S0)'] < hw6discreteDividendModel.S0 + hw6FD.deltaX * 1.5)]
df_display
```

Out[10]:

	Spot Price (S0)	Forward Price (X0)	Call Price
999	100.098758	99.34782	5.827586
1000	100.000000	99.24782	5.775510
1001	99.901242	99.14782	5.723719